Gemini Recommendations from Timing Reports

Based on an exhaustive analysis of the execution logs and Framework Timing Reports (v1.6.14), here is a prioritized engineering review and actionable roadmap for Master List Framework v1.6.15.

Executive Summary & Critical Metrics

Process / Workflow	createMasterList	formatRawData	runDashboardQualityFull	processDemoP	Primary Bottleneck	Target Runtime
Current Runtime	Status / Severity	Primary Bottleneck	Target Runtime	**CRITICAL FAILURE**	**Fatal TypeError reading getLastRow of undefined inside trimOutputSheetToDataSize_.**	**< 15.00s**
61.37s (Crashed)	**CRITICAL FAILURE**	Fatal TypeError reading getLastRow c	**< 15.00s**	**CRITICAL**	**Redundant grid copies (166s) + writing 320,000 cells just to update 218 Banner rows (101s).**	**< 45.00s**
340.68s – 599.18s	**CRITICAL**	Redundant grid copies (166s) + writin	**< 45.00s**	**CRITICAL**	**Repeated structural row shifts and styling re-computations executed sequentially per section.**	**< 40.00s**
287.00s – 394.51s	**CRITICAL**	Repeated structural row shifts and st	**< 40.00s**	**BOTTLENECK**	**Double read/write cycle: writes raw mapped data (33.78s) then re-reads and overwrites entire grid (18.86s).**	**< 20.00s**

1. CRITICAL PRIORITY: Resolve Fatal Crash in createMasterList

Diagnostic Root Cause

In execution Jul 6, 2026, 9:14:40 AM, createMasterList failed after 61.371s with the following fatal trace:

Cannot read properties of undefined (reading 'getLastRow') at buildDashboardSectionIndex (Code: 3335:34) at readDashboardSectionRows (Code: 3605:52) at loadGlobalSettings_ (Code: 3357:16) at trimOutputSheetToDataSize_ (Code: 1312:19)

Inside createMasterList, the script invokes trimOutputSheetToDataSize_(masterSheet, copiedRowCount || 0, SHEET_TYPE.MASTER_LIST).

Inside trimOutputSheetToDataSize_, the function calls const globals = loadGlobalSettings_(); without arguments.

loadGlobalSettings_(sheet) expects a sheet or index object. Because zero arguments are passed, sheet evaluates to undefined.

readDashboardSectionRows_(undefined, RFF_SECTION_GLOBAL) passes undefined to buildDashboardSectionIndex_ (undefined), which executes undefined.getLastRow() and crashes the script.

Code Remediation

Update loadGlobalSettings_ to defensively fall back to fetching the active dashboard sheet when invoked without arguments:

JavaScript

```
function loadGlobalSettings_(sheetOrIndex) {
// Add defensive fallback if invoked without arguments
if (!sheetOrIndex) {
const ss = SpreadsheetApp.getActiveSpreadsheet();
```

```
sheetOrIndex = ss.getSheetByName(RFF_DASHBOARD_SHEET);
if (!sheetOrIndex) throw new Error("Format Dashboard sheet not found.");
}
const rows = readDashboardSectionRows_(sheetOrIndex, RFF_SECTION_GLOBAL);
// ... rest of existing loadGlobalSettings_ logic
}
```

2. PERFORMANCE BOTTLENECK: formatRawData (Shedding ~4.5 Minutes)

Format Raw Data currently consumes 5.6 to 10 minutes (340.68s – 599.18s). Two steps account for over 80% of this runtime:

A. Eliminate Redundant Template Cloning Overhead (161s – 166s)

Finding: Copy Template - Raw Data and paste mapped source values consumes 161.57s to 166.46s (~2.7 minutes).

Root Cause: In createRawDataOutputSheetFromTemplateFast_, template.copyTo(ss) already duplicates the entire sheet (grid dimensions, column widths, formatting, and frozen rows). However, the script subsequently executes resizeSheetGrid_, performs explicit .copyTo() operations for title and data formatting rows, and executes a for loop calling setColumnWidth across 54 columns. On a 6,500-row sheet, forcing Google Sheets to re-apply structural grid operations over duplicated cells causes extreme backend layout recalculations.

Remediation: Remove redundant grid sizing and formatting copies after template.copyTo(ss). Simply duplicate the template, clear target data cells if necessary, and write values:

JavaScript

```
function createRawDataOutputSheetFromTemplateFast_(sheetType, outputName, outputRows, firstDay, endDate, sourceName) {
const ss = SpreadsheetApp.getActiveSpreadsheet();
const dashboard = loadDashboardConfig_();
const sheetDef = getSheetDefinitionByType_(dashboard, sheetType);
const template = ss.getSheetByName(sheetDef.templateName);
deleteSheetIfExists_(ss, outputName, sourceName, sheetDef.templateName);
// 1. Single structural cloning operation
const sheet = template.copyTo(ss);
sheet.showSheet();
setUniqueSheetName_(sheet, outputName);
ss.setActiveSheet(sheet);
const width = Math.max(outputRows[0] ? outputRows[0].length : 1, 1);
const requiredRows = Math.max(DATA_START_ROW + outputRows.length - 1, DATA_START_ROW);
// Only expand rows if output exceeds template capacity
if (sheet.getMaxRows() < requiredRows) {
sheet.insertRowsAfter(sheet.getMaxRows(), requiredRows - sheet.getMaxRows());
}
// 2. Write Metadata and Values
sheet.getRange("A1").setValue(sheetDef.reportTitle || outputName);
if (firstDay) sheet.getRange("B2").setValue(firstDay);
if (endDate) sheet.getRange("D2").setValue(endDate);
if (outputRows.length) {
sheet.getRange(DATA_START_ROW, 1, outputRows.length, width).setValues(outputRows);
}
clearSheetRuntimeCachesForSheet_(sheet);
```

return sheet;
}
B. Eliminate Massive I/O Overwrite in Banner Sync (101s – 110s)
Finding: Sync Raw Data Banner fields from Banners sheet - rows updated: 215 takes 101.13s to 110.16s (~1.8 minutes).
Root Cause: syncRawDataBannerColumns_ reads all ~6,000 rows x 54 columns into memory. It updates 215 rows in memory, but then executes range.setValues(values) across the entire 324,000-cell range to write back those few updates.
Remediation: Merge Banner field synchronization directly into the initial memory mapping inside formatRawData() before the sheet is populated. By mapping Banner fields in memory while constructing outputRows, you drop the 101-second API write overhead to 0.00s.
3. PERFORMANCE BOTTLENECK: runDashboardQualityFull (Shedding ~3.5 Minutes)
Dashboard Quality Workflow consumes 286.99s to 394.51s (~5 to 6.5 minutes).
Finding: When executed standalone, saving Section I Framework Health Check takes 18.95s and saving Section L Performance Summary takes 20.03s. However, during the sequential Dashboard Quality Workflow, Section I degrades to 75.86s, Section L degrades to 66.64s, and Section N degrades to 56.90s.
modifications on a 1,000-row formatted report forces Google Sheets to repeatedly re-index borders, merges, and conditional formatting rules.
Remediation: Refactor runDashboardQualityFull() to run in deferred write mode. Execute all evaluation logic in memory, persist the arrays to PropertiesService, and execute a single unified structural update (replaceDashboardQualitySectionsRows_) at the end of the run:
JavaScript
function runDashboardQualityFull() {
return runFrameworkTimed_("Dashboard Quality Workflow", function(timing) {
ensureDashboardQualityReportSheet_();
// 1. Evaluate all sections in memory without modifying spreadsheet grid
const healthRows = collectFrameworkHealthCheckRows_();
saveDashboardQualitySectionRows_(RFF_HEALTH_CHECK_SHEET, healthRows, { deferSheetWrite: true });
markFrameworkStep_(timing, "Section I Framework Health Check evaluated");
const perfRows = collectDashboardQualityPerformanceSummaryRows_();
saveDashboardQualitySectionRows_(RFF_PERFORMANCE_SUMMARY_KEY, perfRows, { deferSheetWrite: true });
markFrameworkStep_(timing, "Section L Performance Summary evaluated");
const syncRows = collectWorkflowSyncVerificationRows_();
saveDashboardQualitySectionRows_(RFF_WORKFLOW_SYNC_VERIFICATION_KEY, syncRows, { deferSheetWrite: true });
markFrameworkStep_(timing, "Section N Workflow & Synchronization evaluated");
// 2. Execute ONE unified structural write for all sections
const sectionWrites = [
{ title: getDashboardQualitySectionTitleForKey_(RFF_HEALTH_CHECK_SHEET), rows: buildTimestampedDashboardQualitySectionRows_(RFF_HEALTH_CHECK_SHEET, healthRows) },
{ title: getDashboardQualitySectionTitleForKey_(RFF_PERFORMANCE_SUMMARY_KEY), rows: buildTimestampedDashboardQualitySectionRows_(RFF_PERFORMANCE_SUMMARY_KEY, perfRows) },
{ title: getDashboardQualitySectionTitleForKey_(RFF_WORKFLOW_SYNC_VERIFICATION_KEY), rows: buildTimestampedDashboardQualitySectionRows_(RFF_WORKFLOW_SYNC_VERIFICATION_KEY, syncRows) }
];

```
replaceDashboardQualitySectionsRows_(ensureDashboardQualityReportSheet_(), sectionWrites);
styleDashboardQualityReport_(ensureDashboardQualityReportSheet_());
writeCombinedFrameworkTimingReport_();
markFrameworkStep_(timing, "Dashboard Quality Sections H-N batch updated");
});
}
```

4. WORKFLOW OPTIMIZATION: processDemoP (Shedding ~40 Seconds)

Process Demo P currently takes 74.43s to 111.22s.

Finding: Step copy active Raw Data rows to Demo P template takes 33.78s, immediately followed by single write processed Demo P data taking 18.86s.

Root Cause: createActiveDemoPFromRawData_ maps raw values and writes them to the sheet via createOutputSheetFromDashboardTemplate_ (setValues). It then immediately invokes processDemoPAsWorkingSource_, which reads the entire 2,118 x 78 grid back out of the sheet (getDataValues_), executes address/phone/contact transformations in memory, and writes the entire grid back to the spreadsheet a second time (range.setValues).

Remediation: Eliminate the double read/write cycle. Pass the in-memory array directly from createActiveDemoPFromRawData_ into the working-source transformation functions before executing the first grid write:

JavaScript

```
// Inside createActiveDemoPFromRawData_:
const workingData = {
headers: headers,
headerMap: buildHeaderIndexMap_(headers),
values: outputRows,
range: null
};
// 1. Perform transformations completely IN MEMORY
updateBannerColumnData_(workingData, null);
combineAddressesData_(workingData, null);
handleLanguageData_(workingData, null);
splitPhoneNumbersData_(workingData, null);
runMasterContactProcessData_(workingData, null);
combineNotesSummaryData_(workingData, null);
populateDemoPUpdateColumns_(workingData, monthParts, rawSheet.getName(), "Created");
populateUniversalMetadataColumns_(workingData, monthParts, rawSheet.getName(), "Demo P", "Created");
// 2. Perform ONE single grid write with fully processed data
const demoSheet = createOutputSheetFromDashboardTemplate_(
SHEET_TYPE.DEMO_P,
targetName,
workingData.values,
monthParts.firstDay,
monthParts.lastDay
);
```

Summary Roadmap for v1.6.15

Immediate Patch: Apply the defensive if (!sheetOrIndex) guard inside loadGlobalSettings_ to prevent createMasterList from crashing on line 3335.

I/O Consolidation: Refactor formatRawData and processDemoP to execute all data mapping and synchronization (Banner sync, address combining, phone splitting) in JavaScript memory prior to invoking .setValues().						
Template Copy Streamlining: Remove post-copy grid dimension loops inside createRawDataOutputSheetFromTemplateFast_.						
Batch QA Execution: Switch runDashboardQualityFull to use deferSheetWrite: true for individual evaluations, committing all structural section updates in a single DOM pass.						